GTU Exam Questions - Solved

Database Management (DI03016041) - GTU Diploma IT Sem 3
Winter 2025 + Summer 2026 papers

What this PDF is: every question from the two GTU papers for this new subject (Winter 2025 and Summer 2026), solved in descriptive, exam-ready form. Each paper has **70 marks**: 5 questions with (a) 3 marks, (b) 4 marks, (c) 7 marks and an **OR** for each part. **SQL and normalization questions are numerical-type - write queries and tables neatly, it is easy full marks.**

PRIORITY 1: Topics that came in BOTH papers (study first)

With only two papers, a topic in both is the safest bet. It is not a guarantee - read the rest too.

Topic	Winter 2025	Summer 2026
Basic terms (metadata, schema, database)	Q1(a) (3M)	Q1(a) (3M)
DBMS advantages / disadvantages	Q1(b) (4M)	Q1(c) OR (7M)
Database Administrator (DBA)	Q1(c) (7M)	Q1(c) (7M)
Data abstraction / ANSI-SPARC	Q1(c) OR (7M)	Q1(b) (4M)
Basic ER terms	Q2(a) (3M), Q2(a) OR (3M)	Q2(a) OR (3M)
Generalization / Specialization	Q2(b) (4M)	Q2(b) (4M), Q2(b) OR (4M)
Attributes and their types	Q2(c) (7M)	Q2(c) OR (7M)
Mapping cardinality	Q2(b) OR (4M)	Q2(a) (3M)
ER diagram and symbols	Q2(c) OR (7M)	Q2(c) (7M)
DDL / DML / DCL / View	Q3(a) (3M), Q3(a) OR (3M)	Q3(b) (4M), Q3(a) OR (3M), Q3(b) OR (4M)
SQL functions	Q3(b) (4M), Q3(b) OR (4M)	Q3(a) (3M), Q4(a) (3M), Q4(a) OR (3M)
SQL queries on given table	Q3(c) (7M), Q3(c) OR (7M)	Q3(c) (7M)
Functional dependency	Q4(a) (3M), Q5(a) OR (3M)	Q5(b) (4M)
CREATE TABLE with constraints	Q4(b) (4M), Q4(b) OR (4M)	Q5(a) (3M)
2NF	Q4(c) (7M)	Q4(c) OR (7M)
1NF	Q4(c) OR (7M)	Q4(c) (7M), Q4(b) OR (4M)
ACID properties	Q5(b) (4M)	Q5(c) (7M)

Topic	Winter 2025	Summer 2026
Concurrency problems / deadlock	Q5(c) (7M)	Q5(c) OR (7M)
Transaction states	Q5(b) OR (4M)	Q5(b) OR (4M)
Locking / 2PL	Q5(c) OR (7M)	Q5(a) OR (3M)

PRIORITY 2: Asked once (also important)

Topic	Asked in
Normal forms (3NF)	W2025 Q4(a) OR (3M), W2025 Q5(a) (3M)
Constraints and CREATE TABLE	S2026 Q3(c) OR (7M)
Joins	S2026 Q4(b) (4M)

Unit-wise marks pattern (syllabus weightage)

Unit	Weightage	Expected marks out of 70
1 Introduction to Database Systems	16%	≈ 11
2 Entity Relationship Model	20%	≈ 14
3 SQL	25%	≈ 17-18
4 Normalization	19%	≈ 13
5 Transaction Management	20%	≈ 14

Solved paper - Winter 2025 (10-12-2025)

Q1(a) [3 marks] Define: 1) Metadata 2) Schema 3) Database

★ **Repeated topic - also asked in Summer 2026: Q1(a) (3M)**

Term	Definition	Example
Data	Raw, unprocessed facts and figures that have no meaning by themselves	45, 'Ravi', 9876543210
Information	Processed, organised data that has meaning and is useful for decisions	'Ravi scored 45 out of 50 - Grade A'
Database	An organised collection of related data stored together, so that it can be easily accessed, managed and updated	College database: students, courses, marks
DBMS	Software that creates, stores, manages and controls access to the database	Oracle, MySQL, SQL Server, PostgreSQL
Metadata	**'Data about data'** - it describes the structure of the data: table names, column names, data types, sizes, constraints, relationships	Table STUDENT has columns Roll_no NUMBER(5), Name VARCHAR2(30)
Schema	The **overall logical design / structure** of the database (tables, columns, relationships, constraints). It is defined at design time and changes rarely	STUDENT(Roll_no, Name, City, Branch)
Instance	The **actual data stored in the database at a particular moment**. It changes every time data is inserted, updated or deleted	The rows currently in the STUDENT table

Easy way: **Schema = blueprint of a house; Instance = the people living in it today.** Data → (processing) → Information.

Q1(b) [4 marks] Write down disadvantages of Database Management System.

★ **Repeated topic - also asked in Summer 2026: Q1(c) OR (7M)**

1. **High cost** - DBMS software (e.g. Oracle) and powerful hardware are expensive.
2. **Complexity** - designing, setting up and managing a database needs skill and training.
3. **Need for trained staff** - a DBA and programmers must be hired, increasing salary cost.
4. **Size** - DBMS needs a lot of disk space and memory to run.
5. **Performance overhead** - for small, simple applications it can be slower than a plain file system because of the extra features.
6. **Single point of failure** - if the central database or DBMS fails, all applications stop; regular backup is a must.
7. **Security risk** - all data is in one place, so a successful attack can expose everything.
8. **Conversion cost** - moving from an old file system to a DBMS takes time and money.

Q1(c) [7 marks] Explain responsibilities of DBA.

★ **Repeated topic - also asked in Summer 2026: Q1(c) (7M)**

The **Database Administrator (DBA)** is the person (or team) with **central control over the database and the DBMS**. The DBA makes sure the database is available, secure and running efficiently.

Roles and responsibilities of the DBA

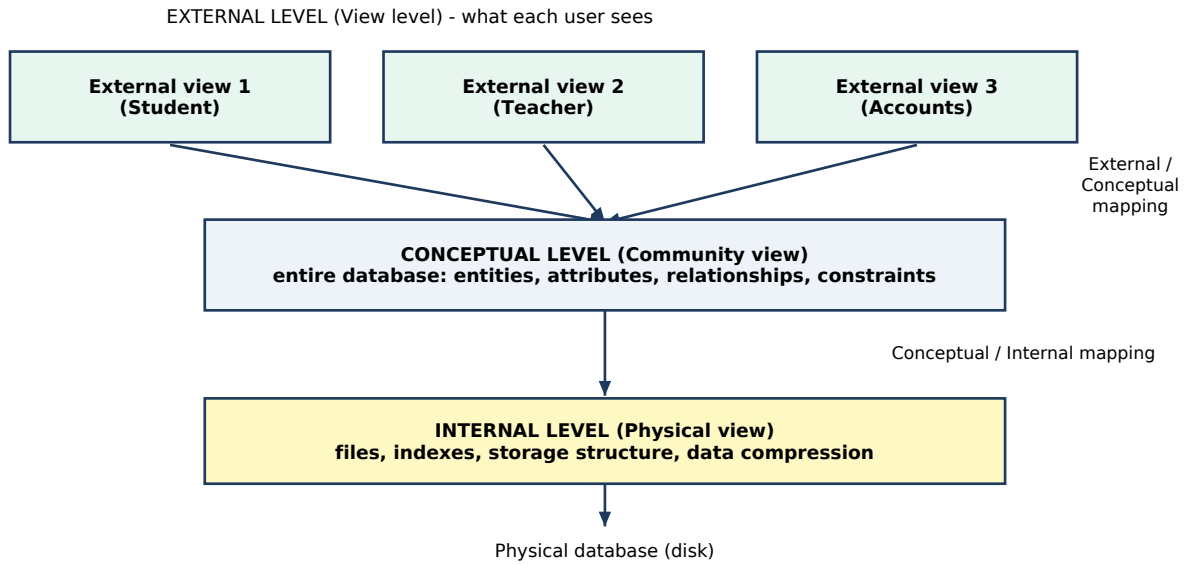
Responsibility	What the DBA does
1. Schema definition	Creates the original database schema using DDL (tables, columns, keys)
2. Storage structure and access method	Decides how data is physically stored, creates indexes for fast access
3. Schema and physical-organisation modification	Changes the schema or storage when needs change (ALTER, reorganise files)
4. Granting authorisation	Decides who can access which data - creates users, roles, GRANT / REVOKE privileges
5. Security and integrity	Protects from unauthorised access; defines integrity constraints so data stays valid
6. Backup and recovery	Takes regular backups; restores data after a crash or disk failure; manages transaction log
7. Performance monitoring and tuning	Watches response time, tunes queries and indexes, manages disk space
8. Routine maintenance	Checks free disk space, installs upgrades / patches, archives old data
9. Communication with users	Talks to users and application programmers, understands requirements, gives training and support
10. Concurrency and data dictionary	Ensures many users can work together safely; maintains the data dictionary

Exam tip: A quick 7-mark answer: intro line + any 7 responsibilities with one line each. Add 'DBA has special privileges (e.g. SYS / SYSTEM in Oracle)'.

Q1(c) OR [7 marks] What is data abstraction? Explain three-level ANSI SPARC architecture in detail.

★ **Repeated topic - also asked in Summer 2026: Q1(b) (4M)**

Data abstraction means **hiding the complex details of how data is stored** and showing the user only what is needed. It makes the system easy to use. The **ANSI-SPARC three-level architecture** gives three levels of abstraction:



Level	Describes	Who uses it	Example
1. Internal (physical) level	**HOW** the data is physically stored on disk: file structure, indexes, record layout, compression, access paths	DBMS and system programmers / DBA	Student file is stored as B-tree index on Roll_no, record of 100 bytes
2. Conceptual (logical) level	**WHAT** data is stored and the relationships among them: all entities, attributes, constraints for the whole database. No storage detail.	DBA and database designers	STUDENT(Roll_no, Name, City, Branch, Fees)
3. External (view) level	The part of the database a **particular user group** sees; many different views possible; hides rest of data	End users, application programmers	Teacher sees (Roll_no, Name, Marks); Accountant sees (Roll_no, Fees)

Mappings: the DBMS converts requests between levels - *external/conceptual mapping* and *conceptual/internal mapping*. This is what gives **data independence**.

Exam tip: 4 marks: write the definition + three levels with one line each. 7 marks: add diagram, table, mappings and data independence.

Q2(a) [3 marks] Define: 1) Relationship 2) Weak Entity Set 3) Primary key

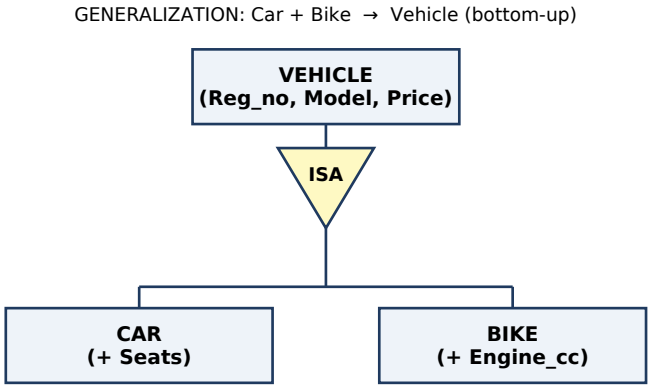
★ Repeated topic - also asked in Summer 2026: Q2(a) OR (3M)

Term	Definition (3-mark style)
Entity	A real-world object or thing, distinguishable from others, about which data is stored (e.g. Student, Book).
Attribute	A property that describes an entity (e.g. Name, Roll_no of Student).
Relationship	An association among two or more entities (e.g. Student *enrolls* in Course).
Recursive relationship set	A relationship in which the same entity set takes part more than once in different roles (e.g. Employee manages Employee).
Weak entity set	Entity set without its own primary key; depends on an owner entity (e.g. Dependent of Employee).
Primary key	The candidate key chosen to uniquely identify each row; cannot be NULL or duplicate.
Super key	A set of one or more attributes that uniquely identifies a row; may contain extra attributes (e.g. {Roll_no, Name}).

Q2(b) [4 marks] Explain Generalization with example.

★ Repeated topic - also asked in Summer 2026: Q2(b) (4M), Q2(b) OR (4M)

Generalization is a **bottom-up** process in which two or more lower-level entity sets that share common attributes are **combined into a higher-level (super class) entity set**.



- Common attributes (Reg_no, Model, Price) are moved up to the super class VEHICLE; special attributes stay with CAR and BIKE.
- The relationship is shown by an **ISA** triangle (CAR *is a* VEHICLE).
- **Purpose:** removes duplication of attributes and shows common features.

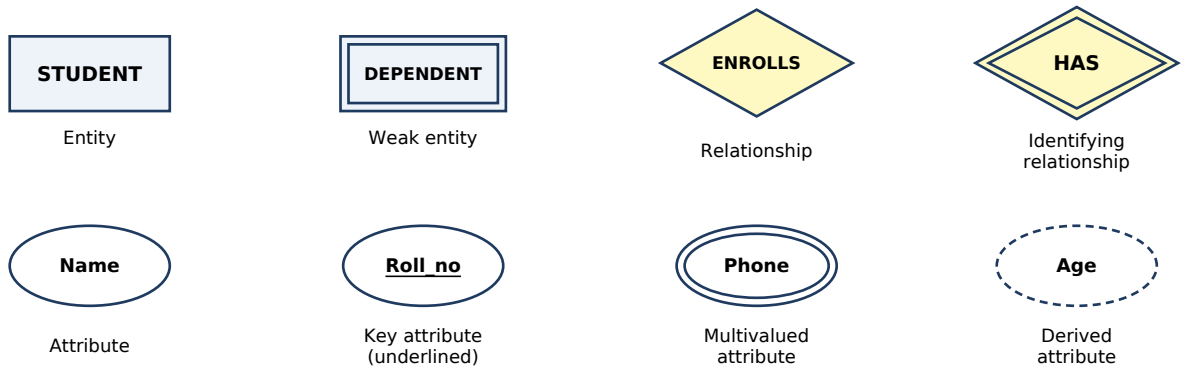
Q2(c) [7 marks] What is attribute? Explain different types of attributes with example.

★ Repeated topic - also asked in Summer 2026: Q2(c) OR (7M)

An **attribute** is a **property or characteristic of an entity (or relationship)**. Each attribute has a **domain** (set of allowed values). Example: for entity STUDENT the attributes are Roll_no, Name, Address, Phone, DOB.

Types of attributes

Type	Meaning	Example	ER symbol
Simple (atomic)	Cannot be divided further	Roll_no, Age, Gender	Single ellipse
Composite	Can be divided into smaller sub-parts	Name → First_name, Middle_name, Last_name; Address → Street, City, Pin	Ellipse with child ellipses
Single-valued	Has only **one value** for an entity	Roll_no, DOB	Single ellipse
Multi-valued	Can have **many values** for one entity	Phone_numbers, Email_ids, Skills	**Double ellipse**
Stored	Value is directly stored in the database	Date_of_birth	Single ellipse
Derived	Value is **calculated from other attribute**; need not be stored	Age (from DOB); Total_marks	**Dashed ellipse**
Key attribute	Uniquely identifies each entity	Roll_no for STUDENT	**Underlined** ellipse
Null attribute	Value unknown or not applicable (NULL)	Middle_name, Passport_no	-
Complex	Combination of composite and multi-valued	Address_history	-



Straight line = links attributes to entities and entities to relationships; 1, M, N near the line show cardinality

Q2(a) OR [3 marks] Define: 1) Entity 2) Recursive Relationship set 3) Super key

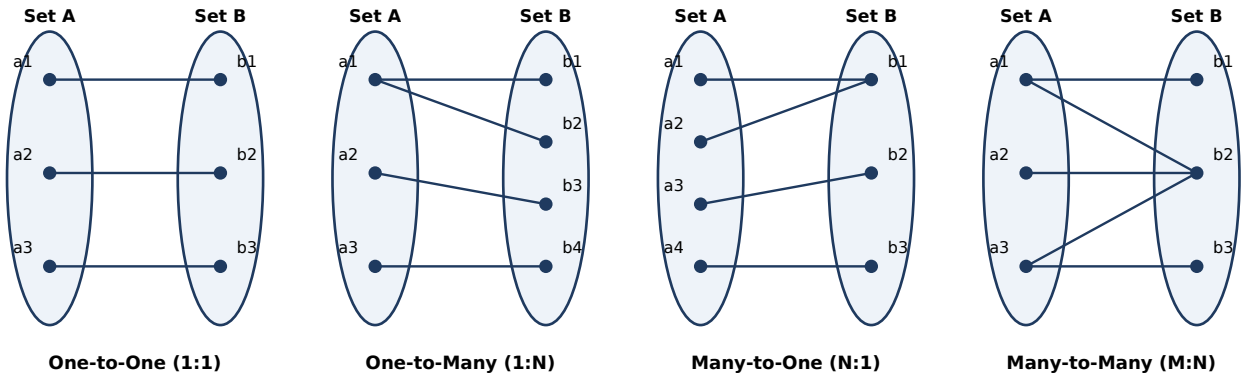
★ Repeated topic - also asked in Summer 2026: Q2(a) OR (3M)

Term	Definition (3-mark style)
Entity	A real-world object or thing, distinguishable from others, about which data is stored (e.g. Student, Book).
Attribute	A property that describes an entity (e.g. Name, Roll_no of Student).
Relationship	An association among two or more entities (e.g. Student *enrolls* in Course).
Recursive relationship set	A relationship in which the same entity set takes part more than once in different roles (e.g. Employee manages Employee).
Weak entity set	Entity set without its own primary key; depends on an owner entity (e.g. Dependent of Employee).
Primary key	The candidate key chosen to uniquely identify each row; cannot be NULL or duplicate.
Super key	A set of one or more attributes that uniquely identifies a row; may contain extra attributes (e.g. {Roll_no, Name}).

Q2(b) OR [4 marks] Explain Mapping Cardinality with suitable example.

★ Repeated topic - also asked in Summer 2026: Q2(a) (3M)

Mapping cardinality (cardinality ratio) tells **how many entities of one set can be associated with how many entities of another set** through a relationship set. For a binary relationship between entity sets A and B there are four types:



Type	Meaning	Example
One-to-One (1:1)	An entity in A is related to **at most one** entity in B and vice versa	Person - Passport; College - Principal
One-to-Many (1:N)	An entity in A can be related to **many** entities in B; an entity in B to only one in A	Department - Students (one department has many students)
Many-to-One (N:1)	Many entities in A relate to **one** entity in B	Employees - Department
Many-to-Many (M:N)	Many in A relate to many in B	Student - Course; Author - Book

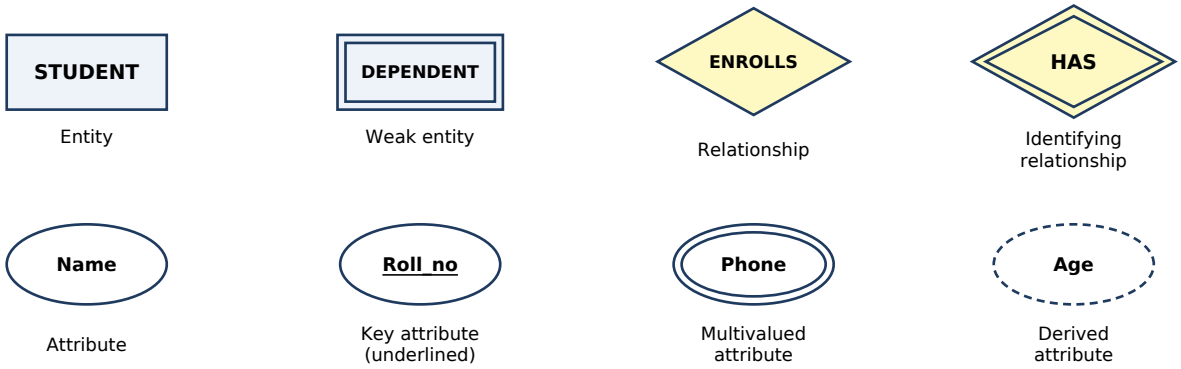
Q2(c) OR [7 marks] What is ER diagram? Explain different symbols used in E-R diagram with example.

★ Repeated topic - also asked in Summer 2026: Q2(c) (7M)

An **E-R diagram** is a **graphical (pictorial) representation** of the entities, their attributes and the relationships among them in a database. It is the blueprint used to design the database.

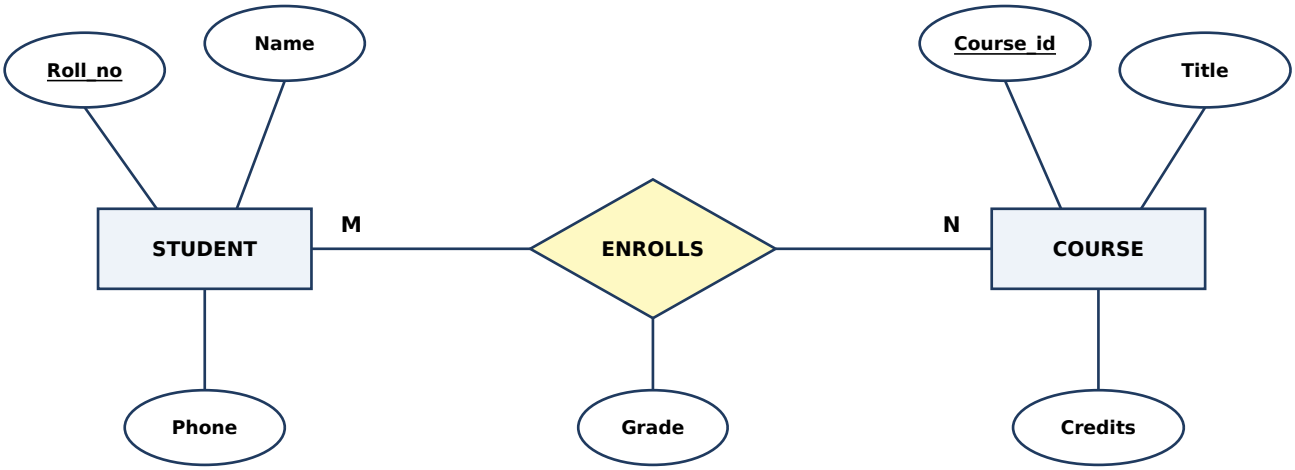
Symbols used in an E-R diagram

Symbol	Name	Meaning
Rectangle	Entity set	A real-world object (STUDENT, COURSE)
Double rectangle	Weak entity set	Entity that has no key of its own and depends on a strong entity (DEPENDENT)
Ellipse (oval)	Attribute	Property of an entity or relationship
Underlined ellipse	Key attribute	Uniquely identifies entities (Roll_no)
Double ellipse	Multi-valued attribute	Can have many values (Phone_no)
Dashed ellipse	Derived attribute	Calculated from other attributes (Age)
Diamond	Relationship set	Association between entities (ENROLLS)
Double diamond	Identifying relationship	Relationship between a weak entity and its owner entity
Line	Link	Connects attributes to entities and entities to relationships
Double line	Total participation	Every entity must participate in the relationship
1, M, N on lines	Cardinality	Shows 1:1, 1:N, M:N



Straight line = links attributes to entities and entities to relationships; 1, M, N near the line show cardinality

Example: Student - Course



Many students enrol in many courses (M : N relationship)

Reading the diagram: entity STUDENT (key Roll_no, attributes Name, Phone) is connected to entity COURSE (key Course_id, attributes Title, Credits) through the relationship ENROLLS. The cardinality is M:N and the relationship has its own attribute Grade.

Q3(a) [3 marks] Define: 1) DDL 2) Grant 3) SQL View

★ Repeated topic - also asked in Summer 2026: Q3(b) (4M), Q3(a) OR (3M), Q3(b) OR (4M)

Term	Definition (3-mark style)
DDL	Data Definition Language - commands that define or change the structure of database objects: CREATE, ALTER, TRUNCATE, DROP, RENAME.
DML	Data Manipulation Language - commands that insert, change and delete the data in tables: INSERT, UPDATE, DELETE.
GRANT	DCL command that gives privileges on a database object to a user. Example: GRANT SELECT, INSERT ON student TO ravi;
REVOKE	DCL command that takes back privileges given earlier. Example: REVOKE INSERT ON student FROM ravi;
SQL View	A virtual table created from a SELECT query; it stores no data of its own. Example: CREATE VIEW it_students AS SELECT * FROM student WHERE branch='IT';
Inner Join	Join that returns only the rows having matching values in both tables. Example: SELECT * FROM student s INNER JOIN department d ON s.dept_id = d.dept_id;

Q3(b) [4 marks] Explain functions: 1) LENGTH 2) MONTHS_BETWEEN

★ Repeated topic - also asked in Summer 2026: Q3(a) (3M), Q4(a) (3M), Q4(a) OR (3M)

How much to write: Write only LENGTH and MONTHS_BETWEEN rows with example.

Character (string) functions

Function	Purpose	Example	Result
LENGTH(s)	**Number of characters** in a string (including spaces)	SELECT LENGTH('DATABASE') FROM DUAL;	8
UPPER(s) / LOWER(s)	Converts case	SELECT UPPER('gtu') FROM DUAL;	GTU
INITCAP(s)	First letter of each word capital	SELECT INITCAP('jay patel') FROM DUAL;	Jay Patel
SUBSTR(s, start, n)	Part of a string	SELECT SUBSTR('DATABASE', 1, 4) FROM DUAL;	DATA
INSTR(s, x)	Position of x in s	SELECT INSTR('DATABASE', 'B') FROM DUAL;	5
CONCAT(a, b) or ||	Joins strings	SELECT 'DB' || 'MS' FROM DUAL;	DBMS
REPLACE(s, a, b)	Replaces text	SELECT REPLACE('cat', 'c', 'b') FROM DUAL;	bat
LPAD / RPAD / TRIM	Pad or remove spaces	SELECT LPAD('5', 3, '0') FROM DUAL;	005

Date functions

Function	Purpose	Example	Result
MONTHS_BETWEEN(d1, d2)	**Number of months between two dates** (d1 later than d2 gives a positive number; can be fractional)	SELECT MONTHS_BETWEEN(TO_DATE('01-07-2025','DD-MM-YYYY'), TO_DATE('01-01-2025','DD-MM-YYYY')) FROM DUAL;	6
SYSDATE	Current system date and time	SELECT SYSDATE FROM DUAL;	today's date
ADD_MONTHS(d, n)	Adds n months to a date	SELECT ADD_MONTHS('15-JAN-2025', 2) FROM DUAL;	15-MAR-2025
LAST_DAY(d)	Last day of the month	SELECT LAST_DAY('10-FEB-2025') FROM DUAL;	28-FEB-2025
NEXT_DAY(d, 'day')	Date of the next given weekday	SELECT NEXT_DAY(SYSDATE, 'MONDAY') FROM DUAL;	next Monday

Q3(c) [7 marks] SQL queries on BOOK_Master(book_id, title, author, price, publisher): insert, Pearson books, title begins with D, add column edition, count rows, descending order of price, drop table.

★ Repeated topic - also asked in Summer 2026: Q3(c) (7M)

BOOK_Master(book_id, title, author, price, publisher)

1. Insert a new book

```
INSERT INTO BOOK_Master (book_id, title, author, price, publisher)
VALUES (101, 'Database System Concepts', 'Korth', 750, 'Pearson');
```

2. Display all books published by 'Pearson'

```
SELECT * FROM BOOK_Master WHERE publisher = 'Pearson';
```

3. Books whose title begins with 'D'

```
SELECT * FROM BOOK_Master WHERE title LIKE 'D%';
```

4. Add a new column edition

```
ALTER TABLE BOOK_Master ADD edition NUMBER(2);
```

5. Count the number of rows

```
SELECT COUNT(*) FROM BOOK_Master;
```

6. All details in descending order of price

```
SELECT * FROM BOOK_Master ORDER BY price DESC;
```

7. Destroy the table along with its data

```
DROP TABLE BOOK_Master;
```

Q3(a) OR [3 marks] Define: 1) DML 2) Revoke 3) Inner Join

★ Repeated topic - also asked in Summer 2026: Q3(b) (4M), Q3(a) OR (3M), Q3(b) OR (4M)

Term	Definition (3-mark style)
DDL	Data Definition Language - commands that define or change the structure of database objects: CREATE, ALTER, TRUNCATE, DROP, RENAME.
DML	Data Manipulation Language - commands that insert, change and delete the data in tables: INSERT, UPDATE, DELETE.
GRANT	DCL command that gives privileges on a database object to a user. Example: GRANT SELECT, INSERT ON student TO ravi;
REVOKE	DCL command that takes back privileges given earlier. Example: REVOKE INSERT ON student FROM ravi;
SQL View	A virtual table created from a SELECT query; it stores no data of its own. Example: CREATE VIEW it_students AS SELECT * FROM student WHERE branch='IT';
Inner Join	Join that returns only the rows having matching values in both tables. Example: SELECT * FROM student s INNER JOIN department d ON s.dept_id = d.dept_id;

Q3(b) OR [4 marks] Explain aggregate functions: 1) MIN 2) COUNT

★ Repeated topic - also asked in Summer 2026: Q3(a) (3M), Q4(a) (3M), Q4(a) OR (3M)

How much to write: Write only MIN and COUNT rows with example.

Aggregate (group) functions work on a set of rows and return **one single value**. They ignore NULL values (except COUNT(*)).

Table **EMP(emp_id, emp_name, dept, salary)**: salaries 60000, 40000, 50000, 45000, 35000 (5 rows).

Function	Purpose	Example query	Result
MIN()	Smallest value	SELECT MIN(salary) FROM emp;	35000
MAX()	Largest value	SELECT MAX(salary) FROM emp;	60000
COUNT()	Number of rows. COUNT(*) counts all rows; COUNT(col) counts non-NULL values	SELECT COUNT(*) FROM emp;	5
SUM()	Total of a numeric column	SELECT SUM(salary) FROM emp;	230000
AVG()	Average value	SELECT AVG(salary) FROM emp;	46000

Q3(c) OR [7 marks] SQL queries on EMPLOYEE_Master(emp_id, emp_name, dept, salary, hire_date): create, insert, salary > 50,000, delete HR, increase IT salary 10%, add email, total IT salary.

★ Repeated topic - also asked in Summer 2026: Q3(c) (7M)

EMPLOYEE_Master(emp_id, emp_name, dept, salary, hire_date)

1. Create table

```
CREATE TABLE EMPLOYEE_Master (
  emp_id    NUMBER(5) PRIMARY KEY,
  emp_name  VARCHAR2(30),
  dept      VARCHAR2(20),
  salary    NUMBER(10,2),
  hire_date DATE
);
```

2. Insert one record

```
INSERT INTO EMPLOYEE_Master
VALUES (1, 'Ravi Patel', 'IT', 60000, TO_DATE('15-01-2023', 'DD-MM-YYYY'));
```

3. Employees having salary greater than 50,000

```
SELECT * FROM EMPLOYEE_Master WHERE salary > 50000;
```

4. Delete employees of 'HR' department

```
DELETE FROM EMPLOYEE_Master WHERE dept = 'HR';
```

5. Increase salary by 10% for 'IT' department

```
UPDATE EMPLOYEE_Master SET salary = salary + salary * 10 / 100 WHERE dept = 'IT';
-- or: SET salary = salary * 1.10
```

6. Add new field email

```
ALTER TABLE EMPLOYEE_Master ADD email VARCHAR2(50);
```

7. Total salary of all employees of 'IT' department

```
SELECT SUM(salary) FROM EMPLOYEE_Master WHERE dept = 'IT';
```

Q4(a) [3 marks] Explain functional dependency with example.

★ Repeated topic - also asked in Summer 2026: Q5(b) (4M)

How much to write: Write the definition, example and the small table for 3 marks.

Prime and non-prime attributes

- **Prime attribute:** an attribute that is **part of some candidate key**.
- **Non-prime attribute:** an attribute that is **not part of any candidate key**.

Functional dependency (FD)

A **functional dependency X → Y** means: if two rows have the **same value of X, they must have the same value of Y**. We say 'X determines Y' (Y is functionally dependent on X); X is the **determinant**.

Example: In STUDENT(Roll_no, Name, City), **Roll_no → Name** and **Roll_no → City**, because one roll number gives exactly one name and city. But Name → Roll_no is not an FD, because two students may have the same name.

Roll_no	Name	City
101	Ravi	Surat
102	Meera	Rajkot
103	Ravi	Pune

Roll_no → Name holds (each roll no. has one name). Name → City does not hold (Ravi has Surat and Pune).

Types of functional dependency

Type	Meaning	Example
Full functional dependency	Y depends on the **whole** composite key X, and not on any proper part of X	(Roll_no, Course_id) → Marks
Partial dependency	Y depends on **only a part** of a composite key	(Roll_no, Course_id) → Name, but Roll_no → Name alone
Transitive dependency	A non-key attribute depends on another non-key attribute: X → Y and Y → Z, so X → Z	Emp_id → Dept_id, Dept_id → Dept_name, so Emp_id → Dept_name

Q4(b) [4 marks] Create STUDENT table: stu_id primary key; branch must be IT, Computer, Electrical or Civil.

★ Repeated topic - also asked in Summer 2026: Q5(a) (3M)

```
CREATE TABLE STUDENT (
  stu_id      NUMBER(5) PRIMARY KEY,
  stu_name    VARCHAR2(30),
  birthdate   DATE,
  City        VARCHAR2(20),
  contact_no  NUMBER(10),
  Branch_name VARCHAR2(15)
              CHECK (Branch_name IN ('IT', 'Computer', 'Electrical', 'Civil'))
);
```

(i) PRIMARY KEY on stu_id makes it unique and not null. (ii) CHECK ... IN (...) allows only the four branch names.

Q4(c) [7 marks] Doctor_Patient relation: a) prime and non-prime attributes b) functional dependencies c) convert to 2NF.

★ Repeated topic - also asked in Summer 2026: Q4(c) OR (7M)

Doctor_Patient(PatientID, DoctorID, PatientName, DoctorName, Department)

PatientID	DoctorID	PatientName	DoctorName	Department
P01	D01	Meera	Dr. Patel	Cardiology
P02	D02	Ankit	Dr. Shah	Neurology
P03	D01	Ramesh	Dr. Patel	Cardiology
P01	D03	Meera	Dr. Vasava	Orthopedic

a) Prime and non-prime attributes

PatientID alone repeats (P01 appears twice) and DoctorID alone repeats (D01 appears twice), so neither is a key alone. The **candidate key is (PatientID, DoctorID)**.

- **Prime attributes:** PatientID, DoctorID.
- **Non-prime attributes:** PatientName, DoctorName, Department.

b) Functional dependencies

1. PatientID → PatientName
2. DoctorID → DoctorName
3. DoctorID → Department (so DoctorID → DoctorName, Department)
4. (PatientID, DoctorID) → PatientName, DoctorName, Department (the whole key determines everything)

PatientName depends on part of the key (PatientID) and DoctorName / Department depend on part of the key (DoctorID) → these are **partial dependencies**, so the relation is **not in 2NF**.

c) Conversion to 2NF

Remove partial dependencies by splitting into three tables:

PatientID (PK)	PatientName
P01	Meera
P02	Ankit
P03	Ramesh

PATIENT(PatientID, PatientName)

DoctorID (PK)	DoctorName	Department
D01	Dr. Patel	Cardiology
D02	Dr. Shah	Neurology
D03	Dr. Vasava	Orthopedic

DOCTOR(DoctorID, DoctorName, Department)

PatientID (FK)	DoctorID (FK)
P01	D01
P02	D02
P03	D01
P01	D03

PATIENT_DOCTOR(PatientID, DoctorID) - composite primary key. All three tables are in 2NF (and also in 3NF).

Q4(a) OR [3 marks] Explain 3NF with example.

Third Normal Form (3NF): a table is in 3NF if (i) it is in **2NF** and (ii) it has **no transitive dependency** - no non-prime attribute depends on another non-prime attribute. (Formally: for every FD X → Y, either X is a super key or Y is a prime attribute.)

Example: EMPLOYEE(Emp_id, Emp_name, Dept_id, Dept_name). Emp_id → Dept_id and Dept_id → Dept_name, so Dept_name depends transitively on Emp_id → not in 3NF. **Decompose:** EMPLOYEE(Emp_id, Emp_name, Dept_id) and DEPARTMENT(Dept_id, Dept_name). Now every non-key attribute depends only on the key → 3NF.

Emp_id	Emp_name	Dept_id
1	Ravi	D1
2	Meera	D1
3	Amit	D2

Dept_id	Dept_name
D1	IT
D2	HR

Rule of thumb: 'every non-key attribute must depend on the key, the whole key and nothing but the key.'

Q4(b) OR [4 marks] Create PRODUCT table: prod_name not null; price greater than 0.

★ Repeated topic - also asked in Summer 2026: Q5(a) (3M)

```
CREATE TABLE PRODUCT (
  prod_id    NUMBER(5) PRIMARY KEY,
  prod_name  VARCHAR2(30) NOT NULL,
  price      NUMBER(8,2) CHECK (price > 0),
  mfg_date   DATE
);
```

(i) NOT NULL on prod_name. (ii) CHECK (price > 0) rejects zero or negative prices.

Q4(c) OR [7 marks] Criteria of 1NF? Normalize the given table into 1NF with two different techniques.

★ Repeated topic - also asked in Summer 2026: Q4(c) (7M), Q4(b) OR (4M)

First Normal Form (1NF): a table is in 1NF if it satisfies these **criteria**:

1. Every column contains only **atomic (indivisible, single) values** - no lists, no repeating groups, no multi-valued fields.
2. Each column holds values of **one kind (same domain)**.
3. Each column has a **unique name**; order of rows / columns does not matter.
4. Every row is **unique** (there is a primary key).

Example: converting to 1NF (W2025 Q4c OR)

Not in 1NF - the column Diseases has several values in one cell:

PatientID	PatientName	Diseases
P001	Meera	Fever, Cold
P002	Amit	Diabetes, BloodPressure, Asthma

Technique 1: separate row for each value (repeat the other data)

PatientID	PatientName	Disease
P001	Meera	Fever
P001	Meera	Cold
P002	Amit	Diabetes
P002	Amit	BloodPressure
P002	Amit	Asthma

Primary key becomes (PatientID, Disease). Simple, but PatientName is repeated.

Technique 2: decompose into two tables (separate table for the repeating group)

PATIENT
(PatientID, PatientName)

PATIENT_DISEASE
(PatientID, Disease)

PatientID	PatientName
P001	Meera
P002	Amit

PatientID (FK)	Disease
P001	Fever
P001	Cold
P002	Diabetes
P002	BloodPressure
P002	Asthma

No repetition of patient name; PatientID links the tables. (A third, poor method is adding columns Disease1, Disease2, Disease3 - it wastes space and limits the number of diseases, so it is not recommended.)

Q5(a) [3 marks] Differentiate 3NF vs BCNF.

How much to write: Write the 3NF vs BCNF table; the example is optional.

Boyce-Codd Normal Form (BCNF) is a stronger version of 3NF. A table is in BCNF if it is in 3NF and, for **every functional dependency X → Y, X is a super key**.

Example: TEACHING(Student, Subject, Teacher) with FDs: (Student, Subject) → Teacher and **Teacher → Subject** (each teacher teaches only one subject). Candidate keys: (Student, Subject) and (Student, Teacher). Teacher → Subject violates BCNF because Teacher is not a super key (but it is still in 3NF, since Subject is a prime attribute). **Decompose:** TEACHER_SUBJECT(Teacher, Subject) and STUDENT_TEACHER(Student, Teacher).

3NF vs BCNF

Point	3NF	BCNF
Condition	For X → Y: X is a super key **OR** Y is a prime attribute	For X → Y: X **must be** a super key
Strength	Weaker	Stronger (every BCNF table is in 3NF, not the reverse)
Redundancy	Some may remain	Almost completely removed
Dependency preservation	Always possible	Not always possible
Decomposition	Lossless and dependency preserving	Lossless, may lose some dependencies

Q5(b) [4 marks] Explain ACID properties of transaction.

★ Repeated topic - also asked in Summer 2026: Q5(c) (7M)

ACID properties are the four properties a transaction must have to keep the database **correct and reliable**:

Property	Meaning	Example (transfer Rs. 500 from A to B; A = 2000, B = 1000)	Ensured by
A - Atomicity	**All or nothing**: either all operations of a transaction are done or none	Debit of A and credit of B both happen, or neither. If failure occurs after debit, the debit is rolled back	Transaction management (rollback, log)
C - Consistency	A transaction takes the database from **one valid (consistent) state to another**; all rules / constraints stay true	Before: A + B = 3000. After: A = 1500, B = 1500, total is still 3000	Programmer, integrity constraints
I - Isolation	Concurrent transactions run **as if they were executed one after another**; intermediate results of one are invisible to others	Another transaction reading A and B in between does not see the money 'missing'	Concurrency control (locking)
D - Durability	Once committed, changes are **permanent** even after a crash or power failure	After commit message, A = 1500 and B = 1500 survive a system crash	Recovery system (log, backup)

Exam tip: For 4 marks write the four properties with one meaningful line each. For 7 marks add the transfer example and 'ensured by'.

Q5(c) [7 marks] List down problems of concurrency control. Explain any two in detail with example.

★ Repeated topic - also asked in Summer 2026: Q5(c) OR (7M)

Concurrency control is the management of **simultaneous execution of transactions** so that they do not interfere with each other and the database stays consistent. Without it, these **problems** arise:

1. **Lost update problem**

2. **Dirty read (temporary update / uncommitted dependency) problem**
3. **Unrepeatable read problem**
4. **Incorrect summary (inconsistent analysis) problem**
5. **Phantom read problem**

1) Lost update problem (explained)

Two transactions read the same data item and update it; the update of one is **overwritten and lost**. Balance X = 100. T1 withdraws 10, T2 deposits 20. Correct final value = 110.

Time	T1	T2	Value of X
t1	READ(X) → 100		100
t2		READ(X) → 100	100
t3	X = X - 10 = 90		100
t4		X = X + 20 = 120	100
t5	WRITE(X) → 90		90
t6		WRITE(X) → 120	**120**

Result: X = 120 instead of 110; T1's update (withdrawal) is lost.

2) Dirty read problem (explained)

A transaction reads data written by another transaction that is **not yet committed**; if that transaction later **rolls back**, the first one has used invalid data. X = 100.

Time	T1	T2
t1	X = X + 50; WRITE(X) (X = 150, not committed)	
t2		READ(X) → 150 (dirty value)
t3	ROLLBACK (X back to 100)	
t4		Continues using 150 - **wrong**

Others (one line each)

- **Unrepeatable read:** T1 reads X twice and gets different values because T2 changed X in between.
- **Incorrect summary:** T1 calculates a total (e.g. sum of balances) while T2 is updating some of those values, giving a wrong sum.
- **Phantom read:** T1 runs the same query twice and the second time sees extra rows inserted by T2.

Q5(a) OR [3 marks] Explain transitive functional dependency.

★ Repeated topic - also asked in Summer 2026: Q5(b) (4M)

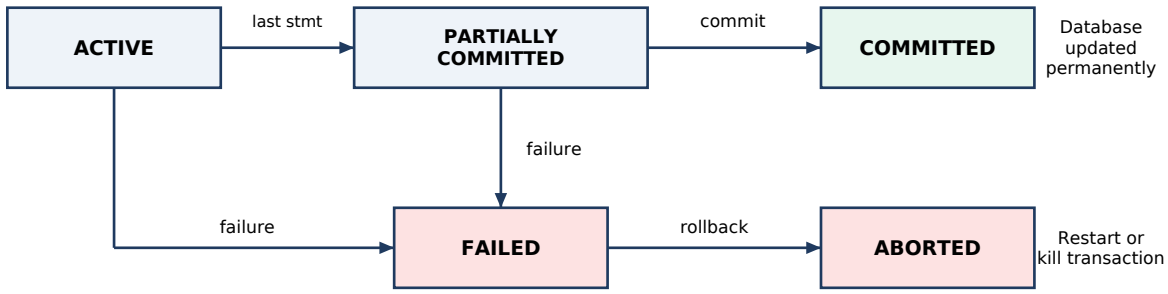
A **transitive dependency** exists when a **non-key attribute depends on another non-key attribute** instead of depending directly on the primary key. If **A → B** and **B → C** (and B does not determine A), then **A → C** is a transitive dependency.

Example: EMPLOYEE(Emp_id, Emp_name, Dept_id, Dept_name). Emp_id → Dept_id and Dept_id → Dept_name, so Emp_id → Dept_name is transitive (Dept_name depends on Emp_id only through Dept_id).

Problem: department name is repeated for every employee of the department → redundancy and anomalies. **Solution:** remove it by converting to 3NF: EMPLOYEE(Emp_id, Emp_name, Dept_id) and DEPARTMENT(Dept_id, Dept_name).

Q5(b) OR [4 marks] Explain state transition diagram of transaction.

★ **Repeated topic - also asked in Summer 2026: Q5(b) OR (4M)**



State	Meaning
Active	Initial state; the transaction is executing its read / write operations
Partially committed	The final statement has been executed, but the changes are still in main memory (not yet permanently on disk)
Committed	The transaction completed successfully and its changes are **permanently stored** in the database
Failed	Normal execution can no longer continue (system error, logical error, power failure)
Aborted	The transaction has been **rolled back** and the database restored to its state before the transaction began. The DBMS may then **restart** the transaction or **kill** it.

Path of a successful transaction: Active → Partially committed → Committed. Path of a failed one: Active (or Partially committed) → Failed → Aborted.

Q5(c) OR [7 marks] What is locking? Explain two phase locking protocol in details with example.

★ **Repeated topic - also asked in Summer 2026: Q5(a) OR (3M)**

Locking is a concurrency-control method in which a transaction must **obtain a lock on a data item before accessing it**, and other transactions have to wait while the item is locked. It guarantees isolation.

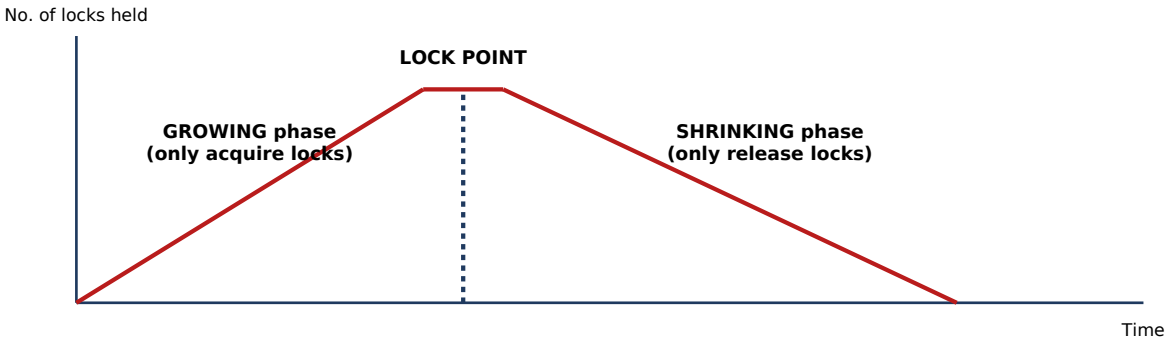
Types of locks

Lock	Purpose	Rule
Shared lock (S) / read lock	For reading a data item	Many transactions can hold S on the same item at once
Exclusive lock (X) / write lock	For reading and writing	Only one transaction can hold X; no other S or X allowed

	Held: S	Held: X
Request S	Granted	Wait
Request X	Wait	Wait

Two-Phase Locking (2PL) protocol

2PL requires every transaction to do its locking in **two phases**:



- **Growing (expanding) phase:** the transaction can **acquire (obtain) locks** but cannot release any lock.
- **Shrinking phase:** the transaction can **release locks** but cannot acquire any new lock.
- The moment the first lock is released is the end of the growing phase (**lock point**). Once it releases a lock, it can never lock again.

2PL guarantees conflict-serializable schedules.

Example

Step	Transaction T1 (transfer from A to B)	Phase
1	LOCK-X(A)	Growing
2	READ(A); A = A - 500; WRITE(A)	Growing
3	LOCK-X(B)	Growing (lock point)
4	READ(B); B = B + 500; WRITE(B)	-
5	UNLOCK(A)	Shrinking
6	UNLOCK(B)	Shrinking

Another transaction T2 that wants A must wait until T1 unlocks A, so T1 and T2 cannot interfere.

Variants: Strict 2PL - all exclusive locks are held until the transaction commits or aborts (avoids cascading rollback). **Rigorous 2PL** - all locks held until commit.

Drawback: 2PL does not prevent **deadlock** and can reduce concurrency.

Solved paper - Summer 2026 (20-05-2026)

Q1(a) [3 marks] Define: 1. Data 2. Information 3. Metadata

★ **Repeated topic - also asked in Winter 2025: Q1(a) (3M)**

Term	Definition	Example
Data	Raw, unprocessed facts and figures that have no meaning by themselves	45, 'Ravi', 9876543210
Information	Processed, organised data that has meaning and is useful for decisions	'Ravi scored 45 out of 50 - Grade A'
Database	An organised collection of related data stored together, so that it can be easily accessed, managed and updated	College database: students, courses, marks
DBMS	Software that creates, stores, manages and controls access to the database	Oracle, MySQL, SQL Server, PostgreSQL
Metadata	**'Data about data'** - it describes the structure of the data: table names, column names, data types, sizes, constraints, relationships	Table STUDENT has columns Roll_no NUMBER(5), Name VARCHAR2(30)
Schema	The **overall logical design / structure** of the database (tables, columns, relationships, constraints). It is defined at design time and changes rarely	STUDENT(Roll_no, Name, City, Branch)
Instance	The **actual data stored in the database at a particular moment**. It changes every time data is inserted, updated or deleted	The rows currently in the STUDENT table

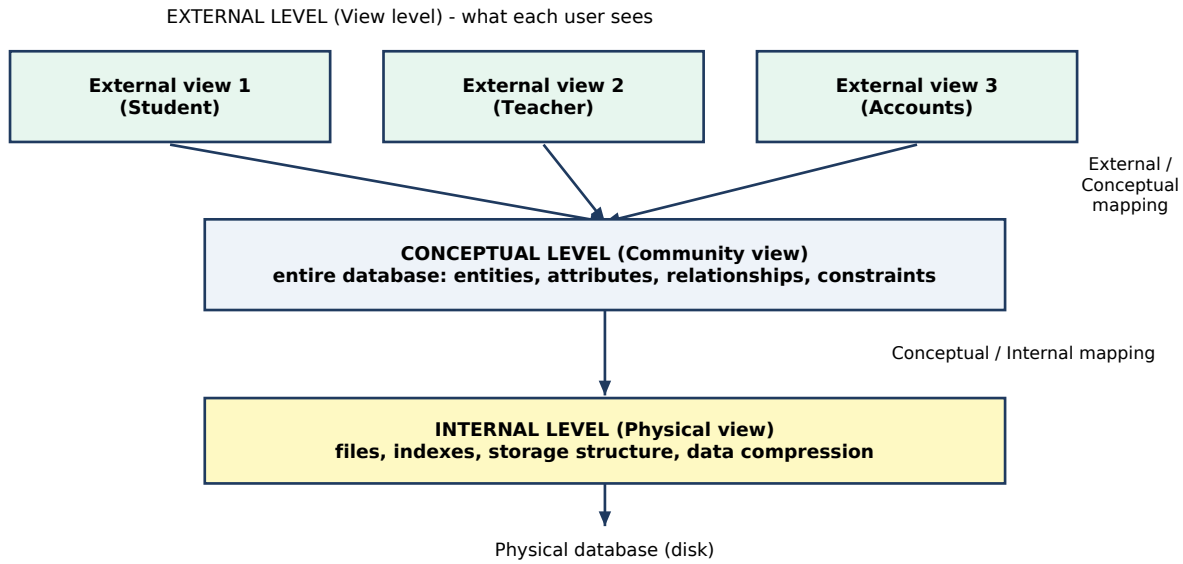
Easy way: **Schema = blueprint of a house; Instance = the people living in it today.** Data → (processing) → Information.

Q1(b) [4 marks] What is data abstraction? Explain 3 levels of data abstraction.

★ **Repeated topic - also asked in Winter 2025: Q1(c) OR (7M)**

How much to write: For 4 marks: definition, the 3 levels (diagram optional).

Data abstraction means **hiding the complex details of how data is stored** and showing the user only what is needed. It makes the system easy to use. The **ANSI-SPARC three-level architecture** gives three levels of abstraction:



Level	Describes	Who uses it	Example
1. Internal (physical) level	**HOW** the data is physically stored on disk: file structure, indexes, record layout, compression, access paths	DBMS and system programmers / DBA	Student file is stored as B-tree index on Roll_no, record of 100 bytes
2. Conceptual (logical) level	**WHAT** data is stored and the relationships among them: all entities, attributes, constraints for the whole database. No storage detail.	DBA and database designers	STUDENT(Roll_no, Name, City, Branch, Fees)
3. External (view) level	The part of the database a **particular user group** sees; many different views possible; hides rest of data	End users, application programmers	Teacher sees (Roll_no, Name, Marks); Accountant sees (Roll_no, Fees)

Mappings: the DBMS converts requests between levels - *external/conceptual mapping* and *conceptual/internal mapping*. This is what gives **data independence**.

Exam tip: 4 marks: write the definition + three levels with one line each. 7 marks: add diagram, table, mappings and data independence.

Q1(c) [7 marks] Describe roles and responsibilities of DBA.

★ Repeated topic - also asked in Winter 2025: Q1(c) (7M)

The **Database Administrator (DBA)** is the person (or team) with **central control over the database and the DBMS**. The DBA makes sure the database is available, secure and running efficiently.

Roles and responsibilities of the DBA

Responsibility	What the DBA does
1. Schema definition	Creates the original database schema using DDL (tables, columns, keys)
2. Storage structure and access method	Decides how data is physically stored, creates indexes for fast access
3. Schema and physical-organisation modification	Changes the schema or storage when needs change (ALTER, reorganise files)
4. Granting authorisation	Decides who can access which data - creates users, roles, GRANT / REVOKE privileges
5. Security and integrity	Protects from unauthorised access; defines integrity constraints so data stays valid
6. Backup and recovery	Takes regular backups; restores data after a crash or disk failure; manages transaction log
7. Performance monitoring and tuning	Watches response time, tunes queries and indexes, manages disk space
8. Routine maintenance	Checks free disk space, installs upgrades / patches, archives old data
9. Communication with users	Talks to users and application programmers, understands requirements, gives training and support
10. Concurrency and data dictionary	Ensures many users can work together safely; maintains the data dictionary

Exam tip: A quick 7-mark answer: intro line + any 7 responsibilities with one line each. Add 'DBA has special privileges (e.g. SYS / SYSTEM in Oracle)'.

Q1(c) OR [7 marks] Explain advantages of DBMS over file system.

★ Repeated topic - also asked in Winter 2025: Q1(b) (4M)

File system stores data in separate files managed by the operating system and application programs. **DBMS** stores data in an integrated database controlled by one software. Advantages of DBMS over a file system:

Point	File-oriented system	Database system (DBMS)	Advantage of DBMS
Redundancy	Same data repeated in many files	Data stored once and shared	Less duplication, saves space
Consistency	Copies get out of date	One copy, updated once	Data is consistent
Data sharing	Difficult; files tied to programs	Many users and applications share one database	Easy sharing
Data independence	Program depends on file format	Program is independent of storage structure	Change storage without changing programs
Integrity	Rules coded in every program	Constraints (PK, FK, CHECK) enforced centrally	Accurate, valid data
Security	Only OS-level, weak	Users, roles, privileges (GRANT / REVOKE)	Controlled access
Concurrency	Not controlled	Locking and transaction management	Safe multi-user access
Backup and recovery	Manual	Automatic backup, log-based recovery	Data safe after a crash
Querying	New program for each query	SQL - powerful query language	Fast, easy retrieval

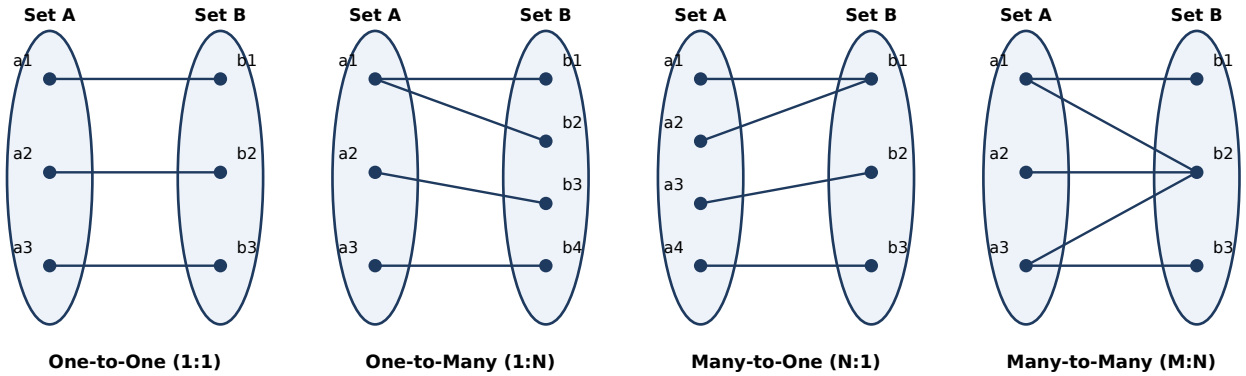
Exam tip: For 7 marks write an introduction of both, then at least 6 advantages with a line each (or the full table).

Q2(a) [3 marks] Explain Mapping Cardinality with diagram.

★ Repeated topic - also asked in Winter 2025: Q2(b) OR (4M)

How much to write: Draw the diagram and write the four types in one line each.

Mapping cardinality (cardinality ratio) tells **how many entities of one set can be associated with how many entities of another set** through a relationship set. For a binary relationship between entity sets A and B there are four types:

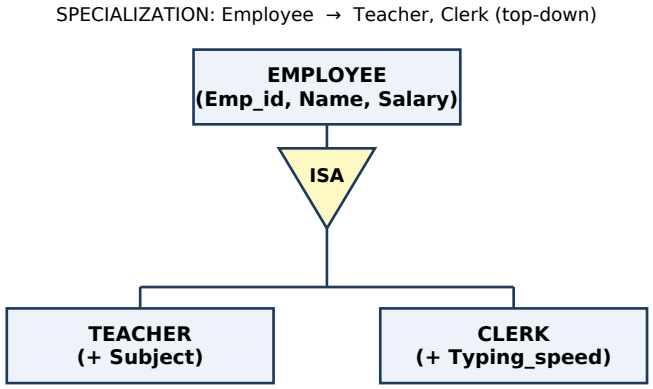


Type	Meaning	Example
One-to-One (1:1)	An entity in A is related to **at most one** entity in B and vice versa	Person - Passport; College - Principal
One-to-Many (1:N)	An entity in A can be related to **many** entities in B; an entity in B to only one in A	Department - Students (one department has many students)
Many-to-One (N:1)	Many entities in A relate to **one** entity in B	Employees - Department
Many-to-Many (M:N)	Many in A relate to many in B	Student - Course; Author - Book

Q2(b) [4 marks] Explain Specialization with example.

★ Repeated topic - also asked in Winter 2025: Q2(b) (4M)

Specialization is a **top-down** process in which a **higher-level entity set is divided into lower-level (sub class) entity sets** based on some distinguishing characteristic.



- Sub classes **inherit all attributes** of the super class and add their own specific attributes.
- **Example:** EMPLOYEE is specialised into TEACHER (extra attribute Subject) and CLERK (extra attribute Typing_speed).
- **Generalization vs specialization:** opposite processes - generalization is bottom-up (combine), specialization is top-down (split).

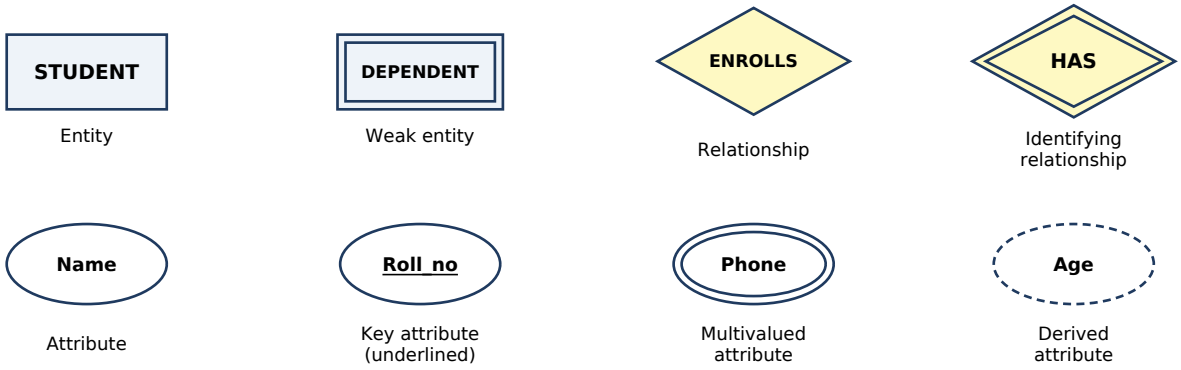
Q2(c) [7 marks] Define E-R diagram. Explain different symbols used in E-R diagram with example.

★ Repeated topic - also asked in Winter 2025: Q2(c) OR (7M)

An **E-R diagram** is a **graphical (pictorial) representation** of the entities, their attributes and the relationships among them in a database. It is the blueprint used to design the database.

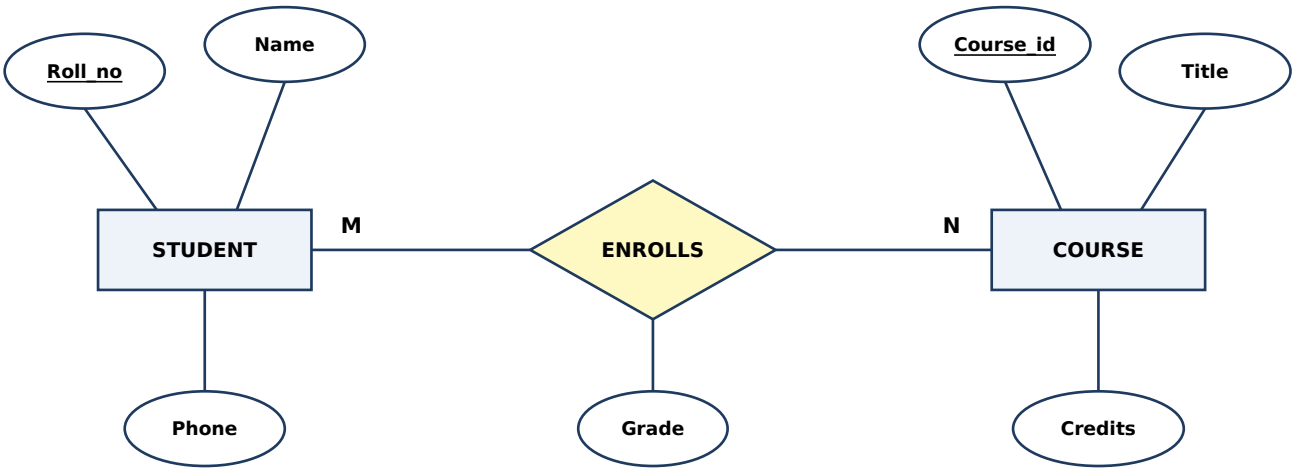
Symbols used in an E-R diagram

Symbol	Name	Meaning
Rectangle	Entity set	A real-world object (STUDENT, COURSE)
Double rectangle	Weak entity set	Entity that has no key of its own and depends on a strong entity (DEPENDENT)
Ellipse (oval)	Attribute	Property of an entity or relationship
Underlined ellipse	Key attribute	Uniquely identifies entities (Roll_no)
Double ellipse	Multi-valued attribute	Can have many values (Phone_no)
Dashed ellipse	Derived attribute	Calculated from other attributes (Age)
Diamond	Relationship set	Association between entities (ENROLLS)
Double diamond	Identifying relationship	Relationship between a weak entity and its owner entity
Line	Link	Connects attributes to entities and entities to relationships
Double line	Total participation	Every entity must participate in the relationship
1, M, N on lines	Cardinality	Shows 1:1, 1:N, M:N



Straight line = links attributes to entities and entities to relationships; 1, M, N near the line show cardinality

Example: Student - Course



Many students enrol in many courses (M : N relationship)

Reading the diagram: entity STUDENT (key Roll_no, attributes Name, Phone) is connected to entity COURSE (key Course_id, attributes Title, Credits) through the relationship ENROLLS. The cardinality is M:N and the relationship has its own attribute Grade.

Q2(a) OR [3 marks] Define: 1. Entity 2. Attributes 3. Relationship

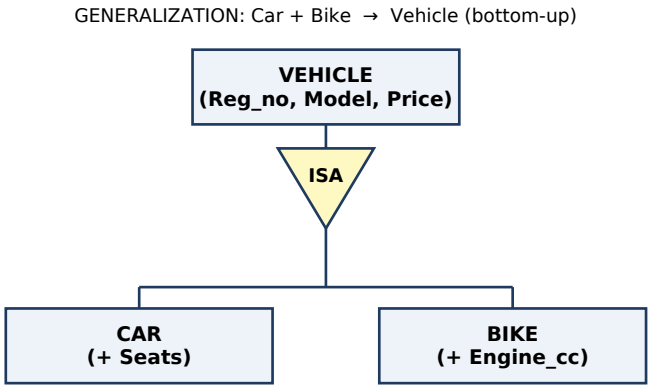
★ Repeated topic - also asked in Winter 2025: Q2(a) (3M), Q2(a) OR (3M)

Term	Definition (3-mark style)
Entity	A real-world object or thing, distinguishable from others, about which data is stored (e.g. Student, Book).
Attribute	A property that describes an entity (e.g. Name, Roll_no of Student).
Relationship	An association among two or more entities (e.g. Student *enrolls* in Course).
Recursive relationship set	A relationship in which the same entity set takes part more than once in different roles (e.g. Employee manages Employee).
Weak entity set	Entity set without its own primary key; depends on an owner entity (e.g. Dependent of Employee).
Primary key	The candidate key chosen to uniquely identify each row; cannot be NULL or duplicate.
Super key	A set of one or more attributes that uniquely identifies a row; may contain extra attributes (e.g. {Roll_no, Name}).

Q2(b) OR [4 marks] Describe Generalization.

★ Repeated topic - also asked in Winter 2025: Q2(b) (4M)

Generalization is a **bottom-up** process in which two or more lower-level entity sets that share common attributes are **combined into a higher-level (super class) entity set**.



- Common attributes (Reg_no, Model, Price) are moved up to the super class VEHICLE; special attributes stay with CAR and BIKE.
- The relationship is shown by an **ISA** triangle (CAR *is a* VEHICLE).
- **Purpose:** removes duplication of attributes and shows common features.

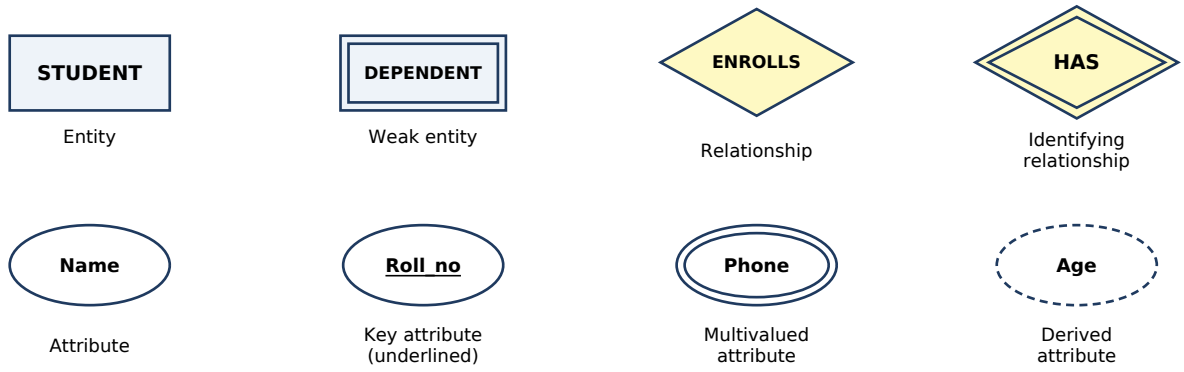
Q2(c) OR [7 marks] Define attribute. Explain different types of attributes with example.

★ Repeated topic - also asked in Winter 2025: Q2(c) (7M)

An **attribute** is a **property or characteristic of an entity (or relationship)**. Each attribute has a **domain** (set of allowed values). Example: for entity STUDENT the attributes are Roll_no, Name, Address, Phone, DOB.

Types of attributes

Type	Meaning	Example	ER symbol
Simple (atomic)	Cannot be divided further	Roll_no, Age, Gender	Single ellipse
Composite	Can be divided into smaller sub-parts	Name → First_name, Middle_name, Last_name; Address → Street, City, Pin	Ellipse with child ellipses
Single-valued	Has only **one value** for an entity	Roll_no, DOB	Single ellipse
Multi-valued	Can have **many values** for one entity	Phone_numbers, Email_ids, Skills	**Double ellipse**
Stored	Value is directly stored in the database	Date_of_birth	Single ellipse
Derived	Value is **calculated from other attribute**; need not be stored	Age (from DOB); Total_marks	**Dashed ellipse**
Key attribute	Uniquely identifies each entity	Roll_no for STUDENT	**Underlined** ellipse
Null attribute	Value unknown or not applicable (NULL)	Middle_name, Passport_no	-
Complex	Combination of composite and multi-valued	Address_history	-



Straight line = links attributes to entities and entities to relationships; 1, M, N near the line show cardinality

Q3(a) [3 marks] List any three aggregate functions with example.

★ Repeated topic - also asked in Winter 2025: Q3(b) (4M), Q3(b) OR (4M)

Aggregate (group) functions work on a set of rows and return **one single value**. They ignore NULL values (except COUNT(*)).

Table **EMP(emp_id, emp_name, dept, salary)**: salaries 60000, 40000, 50000, 45000, 35000 (5 rows).

Function	Purpose	Example query	Result
MIN()	Smallest value	SELECT MIN(salary) FROM emp;	35000
MAX()	Largest value	SELECT MAX(salary) FROM emp;	60000
COUNT()	Number of rows. COUNT(*) counts all rows; COUNT(col) counts non-NULL values	SELECT COUNT(*) FROM emp;	5
SUM()	Total of a numeric column	SELECT SUM(salary) FROM emp;	230000
AVG()	Average value	SELECT AVG(salary) FROM emp;	46000

Q3(b) [4 marks] Explain DML commands.

★ Repeated topic - also asked in Winter 2025: Q3(a) (3M), Q3(a) OR (3M)

DML (Data Manipulation Language) commands are used to **insert, modify and delete the data** stored in tables. They can be rolled back (until COMMIT).

1. INSERT - adds new rows

```
INSERT INTO student (roll_no, name, city) VALUES (101, 'Ravi', 'Surat');
INSERT INTO student VALUES (102, 'Meera', 'Rajkot');      -- values for all columns
```

2. UPDATE - changes existing data

```
UPDATE student SET city = 'Vadodara' WHERE roll_no = 101;     -- only one row
UPDATE student SET fees = fees + 500;                           -- all rows (no WHERE)
```

3. DELETE - removes rows

```
DELETE FROM student WHERE roll_no = 102;     -- one row
DELETE FROM student;                             -- all rows, table remains
```

Always use **WHERE** with UPDATE and DELETE, otherwise all rows are affected. SELECT is the DQL command to read data.

Q3(c) [7 marks] SQL statements on Student(Enno, name, branch, sem, clgname, bdate): create, add mobno, insert, IT branch, name begins with 'a', count rows, delete all.

★ Repeated topic - also asked in Winter 2025: Q3(c) (7M), Q3(c) OR (7M)

Student(Enno, name, branch, sem, clgname, bdate)

1. Create table Student

```
CREATE TABLE Student (
  Enno    NUMBER(12) PRIMARY KEY,
  name    VARCHAR2(30),
  branch  VARCHAR2(20),
  sem     NUMBER(1),
  clgname VARCHAR2(50),
  bdate   DATE
);
```

2. Add a column mobno

```
ALTER TABLE Student ADD mobno NUMBER(10);
```

3. Insert one record

```
INSERT INTO Student (Enno, name, branch, sem, clgname, bdate)
VALUES (230001, 'Ankit', 'IT', 3, 'Govt Polytechnic', TO_DATE('12-05-2006', 'DD-MM-YYYY'));
```

4. Students enrolled in 'IT' branch

```
SELECT * FROM Student WHERE branch = 'IT';
```

5. All information where name begins with 'a'

```
SELECT * FROM Student WHERE name LIKE 'a%';
-- ignore case: WHERE LOWER(name) LIKE 'a%'
```

6. Count the number of rows

```
SELECT COUNT(*) FROM Student;
```

7. Delete all records

```
DELETE FROM Student;      -- or TRUNCATE TABLE Student; (faster, cannot be rolled back)
```

Q3(a) OR [3 marks] Explain the GRANT and REVOKE statement in SQL.

★ Repeated topic - also asked in Winter 2025: Q3(a) (3M), Q3(a) OR (3M)

DCL (Data Control Language) commands control **who can access which database objects** - used by the DBA / owner for security.

- **GRANT** - gives privileges (SELECT, INSERT, UPDATE, DELETE, ALL) on an object to a user or role.

```
GRANT SELECT, INSERT ON student TO ravi;
GRANT ALL ON student TO meera WITH GRANT OPTION;   -- meera can pass the privilege on to others
```

- **REVOKE** - takes back privileges previously given.

```
REVOKE INSERT ON student FROM ravi;
REVOKE ALL ON student FROM meera;
```

User management (lab): `CREATE USER ravi IDENTIFIED BY pass123;` `GRANT CREATE SESSION TO ravi;` `DROP USER ravi;`

TCL (for completeness): `COMMIT;` saves changes permanently, `ROLLBACK;` undoes uncommitted changes, `SAVEPOINT sp1;` marks a point to roll back to.

Q3(b) OR [4 marks] Explain DDL commands.

★ Repeated topic - also asked in Winter 2025: Q3(a) (3M), Q3(a) OR (3M)

DDL (Data Definition Language) commands define or change the **structure (schema)** of database objects such as tables. DDL commands are **auto-committed** (cannot be rolled back).

Command	Purpose	Syntax and example
CREATE	Creates a new table (or other object)	CREATE TABLE student (roll_no NUMBER(5), name VARCHAR2(30), city VARCHAR2(20));
ALTER	Changes structure of an existing table: add / modify / drop column	ALTER TABLE student ADD phone NUMBER(10); ALTER TABLE student MODIFY name VARCHAR2(50); ALTER TABLE student DROP COLUMN city;
TRUNCATE	Deletes **all rows** quickly but keeps the table structure; cannot be rolled back	TRUNCATE TABLE student;
DROP	Deletes the **table with its data and structure** permanently	DROP TABLE student;
RENAME	Changes the name of a table	RENAME student TO student_master;

Describe shows the structure of a table: `DESC student;`

TRUNCATE	DELETE	DROP
DDL; removes all rows	DML; removes selected (or all) rows with WHERE	DDL; removes table itself
Cannot rollback, no WHERE	Can rollback, WHERE allowed	Cannot rollback
Structure remains	Structure remains	Structure and data removed

Q3(c) OR [7 marks] Explain SQL constraints.

Constraints are **rules applied on columns of a table** to keep the data **accurate, valid and consistent** (data integrity). The database rejects any INSERT / UPDATE that breaks a rule.

Need of constraints

- Prevent invalid data (negative price, missing name).
- Avoid duplicate rows and keep rows identifiable.
- Maintain correct links between tables (no order for non-existing customer).
- Enforce business rules in the database itself, not only in programs.

Types of constraints

Integrity type	Constraint	Meaning	Example
Domain integrity	**NOT NULL**	Column cannot be empty	name VARCHAR2(30) NOT NULL
	CHECK	Value must satisfy a condition	price NUMBER CHECK (price > 0)
	DEFAULT	Value used when none is given	city VARCHAR2(20) DEFAULT 'Surat'
Entity integrity	**UNIQUE**	No duplicate values (NULL allowed)	email VARCHAR2(50) UNIQUE
	PRIMARY KEY	Unique + NOT NULL; identifies each row; one per table	roll_no NUMBER(5) PRIMARY KEY
Referential integrity	**FOREIGN KEY** (REFERENCES)	Value must exist in the primary / reference key of another table	dept_id NUMBER REFERENCES department(dept_id)

Example with all constraints

```
CREATE TABLE department (
  dept_id   NUMBER(3) PRIMARY KEY,
  dept_name VARCHAR2(20) NOT NULL UNIQUE
);

CREATE TABLE student (
  stu_id   NUMBER(5) PRIMARY KEY,
  stu_name VARCHAR2(30) NOT NULL,
  email    VARCHAR2(40) UNIQUE,
  fees     NUMBER(8,2) CHECK (fees >= 0),
  city     VARCHAR2(20) DEFAULT 'Surat',
  dept_id  NUMBER(3),
  CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

Constraints can be added later: `ALTER TABLE student ADD CONSTRAINT chk_f CHECK (fees >= 0);` and dropped with `ALTER TABLE student DROP CONSTRAINT chk_f;`

Q4(a) [3 marks] List any three numeric functions with example.

★ Repeated topic - also asked in Winter 2025: Q3(b) (4M), Q3(b) OR (4M)

Numeric (single-row) functions take numbers and return numbers. Use the dummy table **DUAL** in Oracle to test them.

Function	Purpose	Example	Result
ABS(n)	Absolute value	SELECT ABS(-15) FROM DUAL;	15
CEIL(n)	Smallest integer ≥ n	SELECT CEIL(4.2) FROM DUAL;	5
FLOOR(n)	Largest integer ≤ n	SELECT FLOOR(4.8) FROM DUAL;	4
ROUND(n, d)	Rounds to d decimal places	SELECT ROUND(45.678, 2) FROM DUAL;	45.68
TRUNC(n, d)	Cuts off after d decimals (no rounding)	SELECT TRUNC(45.678, 1) FROM DUAL;	45.6
MOD(m, n)	Remainder of m ÷ n	SELECT MOD(10, 3) FROM DUAL;	1
POWER(m, n)	m raised to power n	SELECT POWER(2, 3) FROM DUAL;	8
SQRT(n)	Square root	SELECT SQRT(16) FROM DUAL;	4

Q4(b) [4 marks] Explain INNER join with example.

How much to write: Inner join definition + example is enough; other joins are optional.

A **join** combines rows of two or more tables **based on a related column** (usually primary key = foreign key).

INNER JOIN returns **only the rows that have matching values in both tables**; rows without a match are left out.

STUDENT(roll_no, name, dept_id) and **DEPARTMENT**(dept_id, dept_name):

STUDENT
1 Ravi 10
2 Meera 20
3 Amit 30

DEPARTMENT
10 IT
20 Computer
40 Civil

```
SELECT s.roll_no, s.name, d.dept_name
FROM student s INNER JOIN department d
ON s.dept_id = d.dept_id;
```

roll_no	name	dept_name
1	Ravi	IT
2	Meera	Computer

Amit (dept 30) and Civil (40) have no match, so they do not appear.

Other types of joins

Join	Result
LEFT OUTER JOIN	All rows of left table + matching rows of right (NULL where no match)
RIGHT OUTER JOIN	All rows of right table + matching rows of left
FULL OUTER JOIN	All rows of both tables, NULL where no match
CROSS JOIN	Every row of one table with every row of the other (Cartesian product)
SELF JOIN	A table joined with itself (e.g. employee and manager in the same table)

Q4(c) [7 marks] Explain need of normalization. Discuss about 1NF with example.

★ Repeated topic - also asked in Winter 2025: Q4(c) OR (7M)

Normalization is the process of **organising the data of a database into well-structured tables** by decomposing big tables into smaller ones, so that redundancy and anomalies are removed. It is done by applying rules called **normal forms** (1NF, 2NF, 3NF, BCNF).

Importance / need of normalization (S2026 Q4c)

1. **Reduces data redundancy** (same data is not stored again and again).
2. **Removes insertion, update and deletion anomalies.**
3. **Maintains data consistency and integrity.**
4. **Saves storage space** and makes the database easier to maintain.
5. Gives a **logical, clear table structure** and makes queries efficient.

Goals of normalization

- Eliminate redundant (duplicate) data.
- Ensure data dependencies are logical (data stored in the right table).
- Make the design flexible for future changes.
- Minimise anomalies while keeping all information (lossless decomposition).

First Normal Form (1NF): a table is in 1NF if it satisfies these **criteria**:

1. Every column contains only **atomic (indivisible, single) values** - no lists, no repeating groups, no multi-valued fields.
2. Each column holds values of **one kind (same domain)**.
3. Each column has a **unique name**; order of rows / columns does not matter.

4. Every row is **unique** (there is a primary key).

Example: converting to 1NF (W2025 Q4c OR)

Not in 1NF - the column Diseases has several values in one cell:

PatientID	PatientName	Diseases
P001	Meera	Fever, Cold
P002	Amit	Diabetes, BloodPressure, Asthma

Technique 1: separate row for each value (repeat the other data)

PatientID	PatientName	Disease
P001	Meera	Fever
P001	Meera	Cold
P002	Amit	Diabetes
P002	Amit	BloodPressure
P002	Amit	Asthma

Primary key becomes (PatientID, Disease). Simple, but PatientName is repeated.

Technique 2: decompose into two tables (separate table for the repeating group)

PATIENT
(PatientID, PatientName)

PATIENT_DISEASE
(PatientID, Disease)

PatientID	PatientName
P001	Meera
P002	Amit

PatientID (FK)	Disease
P001	Fever
P001	Cold
P002	Diabetes
P002	BloodPressure
P002	Asthma

No repetition of patient name; PatientID links the tables. (A third, poor method is adding columns Disease1, Disease2, Disease3 - it wastes space and limits the number of diseases, so it is not recommended.)

Q4(a) OR [3 marks] Give an example of GROUP BY clause in SQL.

★ Repeated topic - also asked in Winter 2025: Q3(b) (4M), Q3(b) OR (4M)

GROUP BY groups rows that have the same value in the given column(s) into summary rows, so that an **aggregate function** is applied to each group.

Table **EMP**:

emp_id	emp_name	dept	salary
1	Ravi	IT	60000
2	Meera	HR	40000
3	Amit	IT	50000
4	Neha	HR	45000
5	Sunil	Sales	35000

```
SELECT dept, COUNT(*) AS total_emp, SUM(salary) AS total_salary
```

```
FROM emp
GROUP BY dept
HAVING SUM(salary) > 40000;
```

dept	total_emp	total_salary
IT	2	110000
HR	2	85000

Sales (35000) is removed by HAVING. Every non-aggregate column in SELECT must appear in GROUP BY.

Q4(b) OR [4 marks] Convert the Enrollment table (Enrollment No, Student name, Subjects offered: os, java, dm / iits, Python) to 1NF.

★ Repeated topic - also asked in Winter 2025: Q4(c) OR (7M)

Given table (not in 1NF - Subjects offered has many values in one cell):

Enrollment No	Student name	Subjects offered
001	abc	os, java, dm
002	xyz	iits, Python

1NF - one value per cell (one row per subject):

Enrollment No	Student name	Subject offered
001	abc	os
001	abc	java
001	abc	dm
002	xyz	iits
002	xyz	Python

Primary key = (Enrollment No, Subject offered). Now every cell has an atomic value, so the table is in 1NF.

Q4(c) OR [7 marks] Explain 2NF with example.

★ Repeated topic - also asked in Winter 2025: Q4(c) (7M)

Second Normal Form (2NF): a table is in 2NF if (i) it is in **1NF** and (ii) **no non-prime attribute is partially dependent on any candidate key** - i.e. every non-key attribute is **fully functionally dependent** on the whole primary key. (2NF only matters for tables with a **composite key**.)

Example

STUDENT_MARKS(Roll_no, Course_id, Student_name, Course_name, Marks) with primary key (Roll_no, Course_id):

Roll_no	Course_id	Student_name	Course_name	Marks
1	C1	Ravi	Java	80
1	C2	Ravi	SQL	75
2	C1	Meera	Java	90

- Roll_no → Student_name (partial: depends on part of key)
- Course_id → Course_name (partial)
- (Roll_no, Course_id) → Marks (full)

These partial dependencies cause redundancy ('Ravi' and 'Java' repeated). **Decompose to 2NF** by moving each partially dependent attribute with its determinant to a new table:

STUDENT (Roll_no, Student_name)	COURSE (Course_id, Course_name)	MARKS (Roll_no, Course_id, Marks)

Now in every table, each non-key attribute depends on the **whole** key → 2NF.

Q5(a) [3 marks] Create STUDENT table: stu_name not null; branch must be IT, Computer, Electrical or Mechanical.

★ Repeated topic - also asked in Winter 2025: Q4(b) (4M), Q4(b) OR (4M)

```
CREATE TABLE STUDENT (
  stu_id      NUMBER(5) PRIMARY KEY,
  stu_name    VARCHAR2(30) NOT NULL,
  Address     VARCHAR2(50),
  City        VARCHAR2(20),
  contact_no  NUMBER(10),
  Branch_name VARCHAR2(15)
              CHECK (Branch_name IN ('IT', 'Computer', 'Electrical', 'Mechanical'))
);
```

(i) NOT NULL stops empty names. (ii) CHECK allows only 'IT', 'Computer', 'Electrical', 'Mechanical'.

Q5(b) [4 marks] Explain Full Functional Dependency.

★ Repeated topic - also asked in Winter 2025: Q4(a) (3M), Q5(a) OR (3M)

A functional dependency **X → Y** is a **full functional dependency** if Y is functionally dependent on X as a whole, and **not dependent on any proper subset of X**. It matters when X is a composite (multi-attribute) key.

Example: Table MARKS(Roll_no, Course_id, Marks, Student_name). **(Roll_no, Course_id) → Marks** is a full FD: marks depend on both the student and the course; neither Roll_no alone nor Course_id alone can give the marks. But **(Roll_no, Course_id) → Student_name** is **not full** (it is a partial dependency) because Roll_no alone determines Student_name.

- Full FD: removing any attribute from the left side breaks the dependency.
- A table is in **2NF** only when all non-prime attributes are **fully** dependent on the primary key.
- If the key has a single attribute, every FD on it is automatically full.

Q5(c) [7 marks] List and discuss ACID properties of transaction.

★ Repeated topic - also asked in Winter 2025: Q5(b) (4M)

ACID properties are the four properties a transaction must have to keep the database **correct and reliable**:

Property	Meaning	Example (transfer Rs. 500 from A to B; A = 2000, B = 1000)	Ensured by
A - Atomicity	**All or nothing**: either all operations of a transaction are done or none	Debit of A and credit of B both happen, or neither. If failure occurs after debit, the debit is rolled back	Transaction management (rollback, log)
C - Consistency	A transaction takes the database from **one valid (consistent) state to another**; all rules / constraints stay true	Before: A + B = 3000. After: A = 1500, B = 1500, total is still 3000	Programmer, integrity constraints
I - Isolation	Concurrent transactions run **as if they were executed one after another**; intermediate results of one are invisible to others	Another transaction reading A and B in between does not see the money 'missing'	Concurrency control (locking)
D - Durability	Once committed, changes are **permanent** even after a crash or power failure	After commit message, A = 1500 and B = 1500 survive a system crash	Recovery system (log, backup)

Exam tip: For 4 marks write the four properties with one meaningful line each. For 7 marks add the transfer example and 'ensured by'.

Q5(a) OR [3 marks] Explain two phase locking technique.

★ Repeated topic - also asked in Winter 2025: Q5(c) OR (7M)

How much to write: For 3 marks: definition, growing and shrinking phase, one line on lock point.

Locking is a concurrency-control method in which a transaction must **obtain a lock on a data item before accessing it**, and other transactions have to wait while the item is locked. It guarantees isolation.

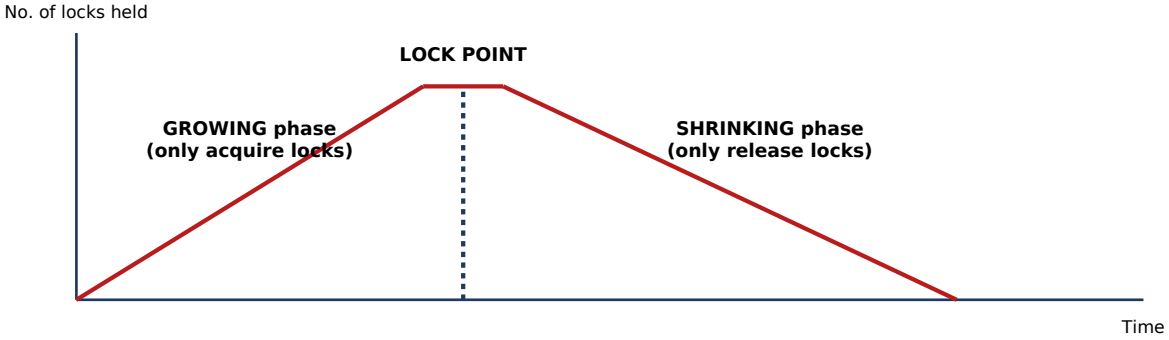
Types of locks

Lock	Purpose	Rule
Shared lock (S) / read lock	For reading a data item	Many transactions can hold S on the same item at once
Exclusive lock (X) / write lock	For reading and writing	Only one transaction can hold X; no other S or X allowed

	Held: S	Held: X
Request S	Granted	Wait
Request X	Wait	Wait

Two-Phase Locking (2PL) protocol

2PL requires every transaction to do its locking in **two phases**:



- **Growing (expanding) phase:** the transaction can **acquire (obtain) locks** but cannot release any lock.
- **Shrinking phase:** the transaction can **release locks** but cannot acquire any new lock.
- The moment the first lock is released is the end of the growing phase (**lock point**). Once it releases a lock, it can never lock again.

2PL guarantees conflict-serializable schedules.

Example

Step	Transaction T1 (transfer from A to B)	Phase
1	LOCK-X(A)	Growing
2	READ(A); A = A - 500; WRITE(A)	Growing
3	LOCK-X(B)	Growing (lock point)
4	READ(B); B = B + 500; WRITE(B)	-
5	UNLOCK(A)	Shrinking
6	UNLOCK(B)	Shrinking

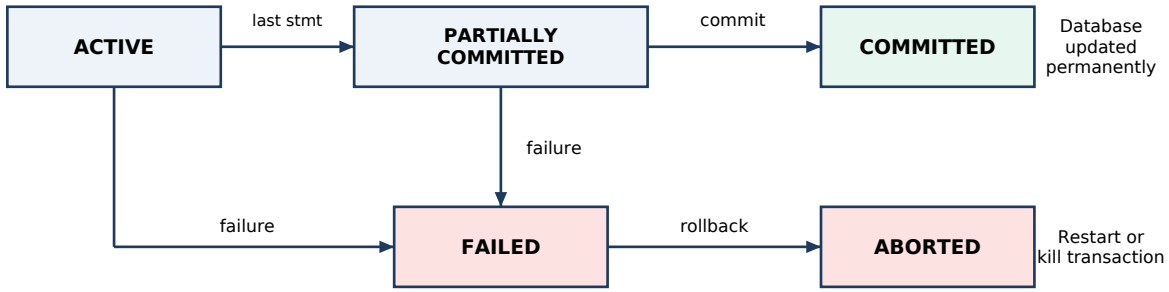
Another transaction T2 that wants A must wait until T1 unlocks A, so T1 and T2 cannot interfere.

Variants: Strict 2PL - all exclusive locks are held until the transaction commits or aborts (avoids cascading rollback). **Rigorous 2PL** - all locks held until commit.

Drawback: 2PL does not prevent **deadlock** and can reduce concurrency.

Q5(b) OR [4 marks] Explain transaction states with proper diagram.

★ Repeated topic - also asked in Winter 2025: Q5(b) OR (4M)



State	Meaning
Active	Initial state; the transaction is executing its read / write operations
Partially committed	The final statement has been executed, but the changes are still in main memory (not yet permanently on disk)
Committed	The transaction completed successfully and its changes are **permanently stored** in the database
Failed	Normal execution can no longer continue (system error, logical error, power failure)
Aborted	The transaction has been **rolled back** and the database restored to its state before the transaction began. The DBMS may then **restart** the transaction or **kill** it.

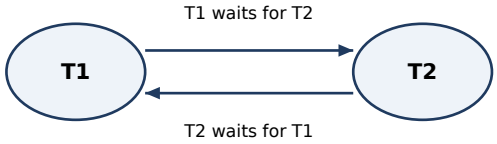
Path of a successful transaction: Active → Partially committed → Committed. Path of a failed one: Active (or Partially committed) → Failed → Aborted.

Q5(c) OR [7 marks] What is deadlock? Explain deadlock detection and prevention.

★ Repeated topic - also asked in Winter 2025: Q5(c) (7M)

A **deadlock** is a situation in which **two or more transactions wait forever**, each waiting for a data item locked by another transaction in the set. None can proceed.

Example: T1 holds a lock on A and requests B; T2 holds a lock on B and requests A. T1 waits for T2 and T2 waits for T1 - deadlock.



Cycle in wait-for graph = DEADLOCK

Deadlock detection

- The DBMS builds a **wait-for graph**: a node for each transaction; an edge Ti → Tj when Ti is waiting for a lock held by Tj.
- The graph is checked periodically. **A cycle in the graph = deadlock.**
- **Recovery:** choose a **victim** transaction (usually the youngest / cheapest to roll back), **abort and roll it back** to release its locks, and restart it later. Avoid always picking the same victim (starvation).

Deadlock prevention

- **Lock all items in advance (conservative 2PL):** a transaction locks every item it needs before it starts; if not all are available, it waits without holding any.
- **Ordering of resources:** all transactions must request locks in a fixed order, so no cycle can form.
- **Timestamp-based schemes** (older transaction has higher priority):
- **Wait-Die:** if an *older* transaction requests an item held by a younger one, it **waits**; if a *younger* requests an item held by an older one, it **dies** (rolls back and restarts).
- **Wound-Wait:** if an older transaction requests an item held by a younger one, it **wounds** (aborts) the younger; a younger one requesting from an older **waits**.
- **Timeout:** a transaction waiting longer than a set time is aborted.

Point	Prevention	Detection
Idea	Make deadlock impossible	Allow it, find it, then break it
Cost	Unnecessary rollbacks, less concurrency	Overhead of checking graph and rollback of victim
Best when	Deadlocks are frequent	Deadlocks are rare